

École Hassania des Travaux Publics (EHTP)

Casablanca, Morocco

Engineering Cycle — Computer Engineering

Detailed Implementation Plan

DevSecOps CI/CD Security & Monitoring Platform

Semgrep — SonarQube — Docker — Kubernetes — Wazuh — ELK — TheHive

Prepared by: Amhidi Hamza

Domain: Cybersecurity — DevSecOps — SOC / Incident Response

Version: 1.0

Date: September 2026

Academic year: 2026–2027

Contents

1	How to Use This Plan	2
2	Phase 0 — Environment and Repository Setup	2
3	Phase 1 — Target Application and Dockerization	3
4	Phase 2 — CI/CD Pipeline Skeleton	3
5	Phase 3 — SAST Integration with Semgrep	4
6	Phase 4 — Code Quality Gate with SonarQube	5
7	Phase 5 — Containerized Build and Kubernetes Deployment	5
8	Phase 6 — Wazuh Runtime Monitoring	6
9	Phase 7 — Elastic Stack Integration	7
10	Phase 8 — Incident Response with TheHive	8
11	Phase 9 — Alert-to-Case Automation (Optional)	8
12	Phase 10 — End-to-End Demonstration and Documentation	9
13	Phase Summary	10

1 How to Use This Plan

The project is broken down into **eleven sequential phases**. Each phase has a stated objective, a prerequisites list, numbered implementation steps, and a validation checkpoint that must pass before moving to the next phase. Phases 0–4 build the shift-left pipeline (code to deployment); phases 5–8 build the shift-right monitoring and response loop; phases 9–10 tie both halves together and produce the final deliverables.

Suggested pace

With the 4–6 week schedule from the specification document, phases 0–2 fit in week 1, phases 3–4 in week 2, phase 5 in week 3, phases 6–7 in week 4, phase 8 in week 5, and phases 9–10 in week 6.

2 Phase 0 — Environment and Repository Setup

Objective

Prepare the local/host environment, version control, and the base directory structure that every later phase will build on.

Prerequisites: a Linux host or VM (or WSL2) with at least 8 GB RAM (16 GB recommended once Wazuh, ELK and TheHive run together), Docker and Docker Compose, `kubectl`, and a Git account.

Steps.

1. Install Docker Engine and Docker Compose; verify with `docker run hello-world`.
2. Install a local Kubernetes cluster tool: `kind` or `minikube`.

```
# kind
curl -Lo ./kind https://kind.sigs.k8s.io/dl/latest/kind-linux-amd64
chmod +x kind && sudo mv kind /usr/local/bin/
kind create cluster --name devsecops-lab
kubectl cluster-info --context kind-devsecops-lab
```

3. Create the Git repository with a directory layout that mirrors the phases:

```
devsecops-platform/
  app/                # target application source
  docker/             # Dockerfile(s)
  k8s/                # Kubernetes manifests
  ci/                 # pipeline configuration
  security/semgrep/   # Semgrep rules/config
  security/sonar/     # sonar-project.properties
  monitoring/wazuh/   # Wazuh manager/agent config
  monitoring/elk/     # Elasticsearch/Logstash/Kibana config
  ir/thehive/         # TheHive/Cortex config, case templates
  docs/              # README, architecture diagram, report
```

4. Create a dedicated Docker network so later stacks (Wazuh, ELK, TheHive) can reach each other without exposing ports publicly:

```
docker network create devsecops-net
```

5. Commit the empty structure with a first README describing the goal and the toolchain.

Validation checkpoint: `docker ps`, `kubectl get nodes` and `git log` all work; the repository skeleton is pushed.

3 Phase 1 — Target Application and Dockerization

Objective

Build (or reuse) a small web application with a handful of intentional, documented vulnerabilities, and containerize it.

Steps.

1. Choose a stack (e.g. Python/Flask or Node/Express) and implement minimal features: login, a protected API endpoint, a database-backed resource lookup.
2. Intentionally introduce a small, documented set of vulnerabilities, for example:
 - a raw SQL query built by string concatenation (SQL injection);
 - an endpoint reflecting user input without escaping (XSS);
 - a hard-coded secret or API key in a config file;
 - an endpoint that returns another user's resource by ID without an ownership check (IDOR-like).
3. Write a docker/Dockerfile using a minimal, pinned base image (e.g. `python:3.12-slim`), a non-root user, and no secrets baked into layers.

```
FROM python:3.12-slim
RUN useradd -m appuser
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
USER appuser
EXPOSE 8080
CMD ["python", "app.py"]
```

4. Build and run the container locally; verify the vulnerable endpoints work as intended (in the isolated lab network only).

```
docker build -t target-app:dev -f docker/Dockerfile .
docker run --rm -p 8080:8080 --network devsecops-net target-app:dev
```

5. Document every intentional vulnerability (file, line, type) in `docs/vulnerabilities.md` — this file will later be checked against Semgrep's findings.

Validation checkpoint: the container starts, the documented vulnerable endpoints are reachable locally, and `docs/vulnerabilities.md` lists them all.

4 Phase 2 — CI/CD Pipeline Skeleton

Objective

Stand up the pipeline that will later host the SAST, quality-gate, build and deploy stages.

Steps.

1. Choose a CI system (GitLab CI, GitHub Actions, or Jenkins) and register a runner with Docker access.
2. Create the top-level pipeline file with empty/placeholder stages so the skeleton is testable end-to-end before adding real tools:

```

stages:
  - sast
  - quality-gate
  - build
  - deploy

sast:
  stage: sast
  script: echo "semgrep placeholder"

quality-gate:
  stage: quality-gate
  script: echo "sonarqube placeholder"

build:
  stage: build
  script: echo "docker build placeholder"

deploy:
  stage: deploy
  script: echo "kubectl apply placeholder"

```

3. Push a commit and confirm all four stages run in order and the pipeline goes green.
4. Add pipeline variables/secrets storage (registry credentials, SonarQube token placeholder, kubeconfig) in the CI system's secret store — never in the repository.

Validation checkpoint: the pipeline runs end-to-end on every push with visible, ordered stages.

5 Phase 3 — SAST Integration with Semgrep

Objective

Replace the SAST placeholder with a real Semgrep scan that fails the pipeline on relevant findings.

Steps.

1. Install Semgrep locally for iteration: `pip install semgrep` (or use the official Docker image `returntocorp/semgrep`).
2. Run a first scan with a broad ruleset to see baseline findings:

```
semgrep --config p/owasp-top-ten app/
```

3. Confirm Semgrep flags the vulnerabilities documented in Phase 1 (SQL injection, XSS, hard-coded secret). Adjust or add custom rules under `security/semgrep/` for anything missed.
4. Wire the real scan into the pipeline stage, exporting JSON and failing on error-level findings:

```

sast:
  stage: sast
  image: returntocorp/semgrep
  script:
    - semgrep --config p/owasp-top-ten --config security/semgrep/
      --error --json -o semgrep.json app/
  artifacts:

```

```
paths: [semgrep.json]
```

5. Verify the pipeline **fails** on the vulnerable commit and **passes** once the vulnerability is fixed — this is the first half of the demonstration scenario.

Validation checkpoint: a deliberately vulnerable commit blocks the pipeline at the sast stage; the fixed commit passes.

6 Phase 4 — Code Quality Gate with SonarQube

Objective

Add a SonarQube quality gate as a second, complementary check (bugs, code smells, security hotspots, coverage).

Steps.

1. Run SonarQube locally via Docker Compose (Community Edition):

```
docker run -d --name sonarqube -p 9000:9000 --network devsecops-net \
  sonarqube:community
```

2. Log in (default admin/admin), change the password, and create a project token for the pipeline.
3. Add sonar-project.properties at the repository root:

```
sonar.projectKey=target-app
sonar.sources=app
sonar.host.url=http://sonarqube:9000
```

4. Configure the Quality Gate conditions (Administration → Quality Gates): e.g. zero new vulnerabilities, security rating A on new code.
5. Wire the scan into the pipeline, waiting for the gate result:

```
quality-gate:
  stage: quality-gate
  image: sonarsource/sonar-scanner-cli
  script:
    - sonar-scanner -Dsonar.login=$SONAR_TOKEN
      -Dsonar.qualitygate.wait=true
```

6. Confirm the pipeline fails when the gate fails and passes once findings are resolved.

Validation checkpoint: SonarQube's dashboard shows the project with an up-to-date analysis, and a failing gate blocks the pipeline exactly like Semgrep does.

7 Phase 5 — Containerized Build and Kubernetes Deployment

Objective

Turn the passing pipeline into a running deployment: build the image, push it, and deploy it to the cluster from Phase 0.

Steps.

1. Stand up (or reuse) a local registry for the lab:

```
docker run -d -p 5000:5000 --network devsecops-net --name registry registry:2
```

2. Implement the build stage: build, tag with the commit SHA, and push.

```
build:
  stage: build
  script:
    - docker build -t localhost:5000/target-app:$CI_COMMIT_SHORT_SHA
      -f docker/Dockerfile .
    - docker push localhost:5000/target-app:$CI_COMMIT_SHORT_SHA
```

3. Write Kubernetes manifests under k8s/: Deployment, Service, and a Secret/ConfigMap for configuration (never hard-coded credentials).

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: target-app
spec:
  replicas: 2
  selector:
    matchLabels: {app: target-app}
  template:
    metadata:
      labels: {app: target-app}
    spec:
      containers:
        - name: target-app
          image: localhost:5000/target-app:latest
          ports: [{containerPort: 8080}]
```

4. Implement the deploy stage, substituting the image tag and applying manifests:

```
deploy:
  stage: deploy
  script:
    - sed -i "s#target-app:latest#target-app:$CI_COMMIT_SHORT_SHA#"
      k8s/deployment.yaml
    - kubectl apply -f k8s/
```

5. Confirm the pods reach Running and the service is reachable inside the cluster/lab network.

Validation checkpoint: `kubectl get pods` shows the application running, deployed automatically by the pipeline after Semgrep and SonarQube both pass.

8 Phase 6 — Wazuh Runtime Monitoring

Objective

Deploy the Wazuh manager and agents to start collecting logs and generating security alerts from the running environment.

Steps.

1. Deploy the Wazuh manager (and its bundled indexer/dashboard, if using the all-in-one Docker Compose distribution) in its own namespace/network, isolated from the application:

```
git clone https://github.com/wazuh/wazuh-docker.git -b v4.9.0
cd wazuh-docker/single-node
docker compose up -d
```

2. Confirm the manager and dashboard are reachable, and change default credentials immediately.
3. Install a Wazuh agent on the Kubernetes node(s) (or as a DaemonSet/sidecar) and on the Docker host running the target application, and register each agent with the manager.

```
# on the host to monitor
curl -so wazuh-agent.deb https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.9.0-1_amd64.deb
WAZUH_MANAGER='<manager-ip>' dpkg -i ./wazuh-agent.deb
systemctl enable wazuh-agent --now
```

4. Enable and configure File Integrity Monitoring (FIM) on sensitive paths (application config, /etc) in `ossec.conf`.
5. Add or enable rules for the two priority detections from the specification: repeated authentication failures (brute force) and unauthorized file changes. Test each by intentionally triggering it (failed logins, editing a monitored file) and confirming an alert appears in the manager's alert log.

Validation checkpoint: `/var/ossec/logs/alerts/alerts.json` on the manager shows alerts for both simulated events, tied to the correct agent.

9 Phase 7 — Elastic Stack Integration

Objective

Centralize Wazuh alerts (and, optionally, application logs) in Elasticsearch and build Kibana dashboards for the SOC analyst.

Steps.

1. If using the Wazuh all-in-one distribution, its bundled indexer (OpenSearch/Elasticsearch-compatible) and dashboard already receive alerts — confirm the `wazuh-alerts-*` index exists and is populated.
2. For a standalone Elastic Stack instead, deploy Elasticsearch, Logstash (or Filebeat) and Kibana via Docker Compose, and configure the Wazuh manager's filebeat output to ship `alerts.json` into Elasticsearch.

```
docker compose -f monitoring/elk/docker-compose.yml up -d
```

3. In Kibana, create an index pattern for the alerts index and verify documents are visible in *Discover*.
4. Build at least three dashboards/visualizations:
 - alert volume over time, split by rule level;
 - top 10 rule descriptions / rule IDs;
 - source IP / affected host table for authentication-failure alerts.
5. Save the dashboards and export them (NDJSON) into `monitoring/elk/` for reproducibility.

Validation checkpoint: triggering the brute-force and FIM test events from Phase 6 makes them visible on the Kibana dashboards within a few seconds to a couple of minutes.

10 Phase 8 — Incident Response with TheHive

Objective

Stand up TheHive and define the case-management workflow that turns a qualified alert into a tracked incident.

Steps.

1. Deploy TheHive (and its Cassandra/Elasticsearch backing services) via Docker Compose:

```
git clone https://github.com/TheHive-Project/TheHive-docker.git
cd TheHive-docker
docker compose up -d
```

2. Log in, create an organisation and analyst accounts, and generate an API key for later automation.
3. Create a **case template** named e.g. wazuh-alert with default tasks: *Triage, Containment, Eradication, Lessons learned*.
4. Manually create one test case from a Phase 6 alert: fill in the title, severity, description, add observables (source IP, username, affected host) and attach the raw alert as an artifact. Walk through the tasks to a closed state to validate the template.
5. Document the manual alert-qualification procedure (who reviews Kibana, when a case is opened, how severity is chosen) in docs/ir-procedure.md. This manual procedure is the required baseline before attempting automation in Phase 9.

Validation checkpoint: a case created from the template can be taken from *New* to *Closed* with a documented resolution, observables attached.

11 Phase 9 — Alert-to-Case Automation (Optional)

Objective

Automate case creation in TheHive for high-severity Wazuh alerts, using the mapping defined in the specification document.

Steps.

1. Decide the integration point: a small polling script (as in the specification's Appendix B pseudo-code) querying the Wazuh/Elasticsearch alerts index, or a Logstash output plugin, or TheHive's own alert-feed feature if available in the deployed version.
2. Implement the connector script under ir/thehive/connector/, using TheHive's REST API (or the thehive4py client library) and the Elasticsearch client for polling:

```
from elasticsearch import Elasticsearch
from thehive4py.api import TheHiveApi
from thehive4py.models import Case, CaseObservable

RULE_LEVEL_THRESHOLD = 12

es = Elasticsearch("http://localhost:9200")
```

```
hive = TheHiveApi("http://localhost:9000", "<API_KEY>")

def poll_alerts():
    query = {"query": {"range": {"rule.level": {"gte": RULE_LEVEL_THRESHOLD}}}}
    return es.search(index="wazuh-alerts-*", body=query)["hits"]["hits"]

def create_case(alert):
    src = alert["_source"]
    case = Case(title=f"Wazuh alert: {src['rule']['description']}",
                tlp=2, severity=3,
                description=str(src), tags=["wazuh", src["rule"]["groups"][0]])
    hive.create_case(case)
```

3. Run the connector on a schedule (cron, or a Kubernetes CronJob) and de-duplicate on alert ID to avoid opening repeat cases.
4. Re-trigger the brute-force test event from Phase 6 and confirm a case appears automatically in TheHive within one polling interval.
5. Tune `RULE_LEVEL_THRESHOLD` and the rule-group mapping to control noise, referencing the severity mapping table from the specification document.

Validation checkpoint: a live simulated attack results in an automatically created, correctly tagged TheHive case with no manual step.

12 Phase 10 — End-to-End Demonstration and Documentation

Objective

Rehearse and record the full before/after scenario, and finalize all deliverables.

Steps.

1. Rehearse the **shift-left half**: commit a vulnerability from `docs/vulnerabilities.md`, show the pipeline blocked at the sast or quality-gate stage, fix it, show the pipeline passing and deploying.
2. Rehearse the **shift-right half**: simulate the attack (e.g. scripted failed-login loop) against the deployed application, show the Wazuh alert, the Kibana dashboard update, and the resulting TheHive case (manual or automated), then walk the case to closure.
3. Record a short screen-capture video or GIF covering both halves end-to-end.
4. Finalize the README with setup instructions reproducing phases 0–9, an architecture diagram, and links to the exported Kibana dashboards and TheHive case template.
5. Write the final report: include the before/after walkthrough, the metrics from the specification's validation-criteria section (detection rate, false-positive tuning notes, pipeline duration, alert-to-case latency), and lessons learned.
6. Tag a release (e.g. v1.0) in the repository once everything above is reproducible from a clean checkout.

Validation checkpoint: a third party can follow the README on a clean environment and reproduce both halves of the demonstration.

13 Phase Summary

Phase	Name	Key output
0	Environment & repository setup	Working Docker/Kubernetes host, repository skeleton
1	Target application & Dockerization	Vulnerable app running in a container
2	CI/CD pipeline skeleton	Four-stage pipeline running end-to-end
3	SAST with Semgrep	Pipeline blocks on vulnerable code
4	Quality gate with SonarQube	Second, independent quality/security gate
5	Build & Kubernetes deployment	Application auto-deployed after both gates pass
6	Wazuh runtime monitoring	Agents reporting, first alerts generated
7	Elastic Stack integration	Kibana dashboards on live alert data
8	Incident response with TheHive	Manual alert-to-case workflow validated
9	Alert-to-case automation (optional)	Automatic case creation from high-severity alerts
10	End-to-end demonstration & docs	Recorded demo, final report, reproducible README

Table 1: Implementation phases at a glance